

CLIFF Interactive Routing

Checkpointed routed execution with shared broadcasts

Simon Frost

Table of contents

Introduction	1
Setup	1
Build the bundled runtime example	2
Run the checkpointed route	2
Inspect the shared broadcast board	3
The pending route is still a semantic artifact	3
Compare with the convergent route	4
Why this matters	4

Introduction

Some CLIFF routes should not run to completion in one uninterrupted pass. A retrieval-heavy Democritus-style route may need a human or downstream agent to approve the retrieval focus before the synthesis stage begins.

This vignette uses the bundled runtime example to show a routed query that stops with an `InteractiveCheckpointRequest` instead of a final artifact.

Setup

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))

using FunctorFlow
```

Build the bundled runtime example

`build_cliff_runtime_example()` registers two demo executors:

- a `company_similarity` route that converges to completion; and
- a `democritus` route that pauses at a checkpoint.

```
example = build_cliff_runtime_example()
runtime = example[:runtime]

println("Registered routes: ", list_route_executors(runtime))
println("Demo queries: ", (example[:company_similarity_query], example[:democritus_query]))
```

Registered routes: [:company_similarity, :democritus]

Demo queries: ("How similar is Adobe to Nike across recent filings?", "Analyze recent studies")

Run the checkpointed route

`execute_cliff_interactive_example()` executes the bundled Democritus query in interactive mode. The route produces evidence and broadcasts, but then returns a `RouteRunResult` with `status == :needs_input`.

```
trace = execute_cliff_interactive_example(example)
summary = summarize_cliff_route_trace(trace)

println("Route: ", summary["route_decision"]["route_name"])
println("Status: ", summary["status"])
println("Counts: ", summary["counts"])
println("Pending checkpoint: ", summary["pending_checkpoint"])
println("Latest workspace: ", summary["latest_workspace"])
println("Latest assessment: ", summary["latest_assessment"])
```

Route: democritus

Status: needs_input

Counts: Dict("broadcasts" => 2, "updates" => 2, "assessments" => 2, "artifacts" => 0, "workspace" => 0)

Pending checkpoint: Dict{String, Any}("request_id" => "democritus-focus-approval", "payload" => Dict("choices" => ["approve", "refine focus"]), "route_name" => "democritus")

Latest workspace: Dict{String, Any}("selected" => Dict{String, Any}("score" => 0.70000000, "focus approval.", "name" => "pending_review", "novelty" => 0.12, "urgency" => 0.88, "relevance" => 0.90, "metadata" => Dict{Any, Any}(...)), "field_of_view" => Dict("capacity" => 4), "remaining" => 0)

Latest assessment: Dict{String, Any}("stability_threshold" => 0.8, "similarity" => nothing, "evidence" => Dict{String, Any}("request_id" => "democritus-focus-approval", "payload" => Dict("choices" => ["approve", "refine focus"]), "route_name" => "democritus"))

The route result is now explicitly blocked on a checkpoint request.

```
println("Needs human input? ", needs_human_input(trace.result))
println("Checkpoint request id: ", trace.result.pending_checkpoint.request_id)
println("Checkpoint prompt: ", trace.result.pending_checkpoint.prompt)
println("Route-local state: ", trace.result.metadata[:route_state])
```

```
Needs human input? true
Checkpoint request id: democritus-focus-approval
Checkpoint prompt: Approve the retrieval focus before the final synthesis step?
Route-local state: Dict{Symbol, Any}(:study_focus => "minimum wage and employment")
```

Inspect the shared broadcast board

The executor published both global and agent-targeted messages. Those can be filtered by audience or tag.

```
println("All broadcast titles: ", broadcast_titles(trace))
println("Messages for synthesis_editor: ", [message.title for message in messages_for_agent(trace)])
println("Checkpoint-tagged messages: ", [message.title for message in messages_for_agent(trace)])
```

```
All broadcast titles: ["Corpus focus proposed", "Checkpoint required"]
Messages for synthesis_editor: ["Corpus focus proposed", "Checkpoint required"]
Checkpoint-tagged messages: ["Checkpoint required"]
```

The pending route is still a semantic artifact

Even a paused route is still represented as a first-class semantic result that can be compiled and inspected.

```
plan = compile_plan(:CLIFFInteractiveVignettePlan, trace.result; metadata=Dict{Symbol, Any}(:example => "cliff"))
ir = lower_plan_to_executable_ir(plan)

println("Compiled subjects: ", [(artifact.subject_name, artifact.subject_kind) for artifact in ir.subjects])
println("IR opcodes: ", [instruction.opcode for instruction in ir.instructions])
```

```
Compiled subjects: [(:democritus__route_run, :cliff_route_run)]
IR opcodes: [:materialize_route_run]
```

Compare with the convergent route

The same bundled runtime also contains a convergent company-similarity route, so the difference between a completed and a checkpointed run is easy to inspect.

```
completed_trace = execute_cliff_runtime_example(example)

println("Completed route status: ", completed_trace.result.status)
println("Completed route artifact count: ", length(completed_trace.result.artifacts))
println("Completed route stop trigger: ", completed_trace.result.convergence.stop_trigger)
```

```
Completed route status: completed
Completed route artifact count: 1
Completed route stop trigger: stability
```

Why this matters

This turns the CLIFF surface into an actual interactive orchestration model:

- routes can pause with explicit checkpoint requests;
- broadcasts remain available while the route is paused;
- the paused result stays compilable and serializable; and
- the same runtime can host both fully convergent and checkpointed routes.

That is a useful foundation for later wiring real `AgentFramework.jl` workflows or GUI-level human-in-the-loop review into the Julia implementation.